Report for ASPIS testing

Corvace Christian, Russo Jacopo

July 10, 2026

Introduction and goals

This document presents the results and analysis of the testing conducted to evaluate the effectiveness of the ASPIS compiler protection. We developed six distinct test programs, each focusing on a specific vulnerability or execution aspect that the compiler is designed to mitigate.

To simulate and inject faults, we utilized GDB by setting breakpoints at strategic execution points. This allowed us to manually alter local variables, data structure fields, pointers, and control registers at runtime. Each program was executed and observed across two compiled versions of the same source code: a standard baseline compiled with Clang, and a protected version compiled with ASPIS, allowing us to clearly document the differences in behavior.

Analysis of single tests

It is important to understand that the creation of these tests required significant trial and error. For our purposes, we determined that a test is considered effective if, following the bit-flip injection, the execution enters a loop within the *DataCorruption_Handler()* function of ASPIS, which represents the intended behavior of the analyzed tool.

Because of this, we made certain assumptions when observing tests that initially failed to trigger the protection. We removed any kind of determinism since with highly predictable machine behavior, the compiler often performs optimizations that inadvertently cause a test to fail.

For example, in this very small C program, the compiler might recognize that the variable n never changes throughout the execution. It could then optimize the compiled assembly code to simply execute a printf instruction, entirely skipping the evaluation or even the allocation of the n variable.

```
#include<stdio.h>

int main(){
    int n=1;
    if (n)
        printf("This is the taken route");
    else
        printf("This is the not taken route");
    return 0;
}
```

Test 1

In this test, we randomly generate an array, sort it using bubble-sort, and then prompt the user to input a number to perform a binary search. The bit-flip injection is executed at line 60, where we modify a number within the randomly generated array. To verify the protection, we then attempt to search for the altered number. This test was designed to observe whether ASPIS would successfully detect modifications to an element within an array. The GDB commands used are listed below; similar command blocks will be found below every test in this document.

```
(gdb) b 60
(gdb) run
(gdb) set var numbers[2]=9999
(gdb) continue
(gdb) Enter the number to search for: 6
```

Test 2

In this test, we calculate a hypothetical pressure using the variables *temp* and *vol* inside a function. We change the value of *temp* before the function's return. If the modification goes through an alarm message would be printed because the pressure would have risen too much. This test was designed to verify how functions would affect ASPIS.

```
(gdb) b 6
(gdb) run
(gdb) set var partial_temp=9999
(gdb) continue
```

Test 3

In this test, we simulate a Kalman filter on 10 random measurements. This test was developed to observe ASPIS' behariour with data in structs, since the filter's parameters are saved in one.

```
(gdb) b 30
(gdb) run
(gdb) set var f.x=9999
(gdb) continue
```

Test 4

In this test, we utilized a for loop containing a variable named *counter* that increments by 2 during each iteration within the loop's body. Additionally, a variable named i is declared and incremented inside the parentheses of the for statement. We performed the bit-flip injection at line 9 for both the i and *counter* variables. This test was designed to verify ASPIS's behavior regarding variables declared both within the loop's initialization statement and inside its body.

```
(gdb) b 9
(gdb) run
(gdb) set var counter=9999
(gdb) continue

(gdb) b 9
(gdb) run
(gdb) set var i=9999
(gdb) continue
```

Test 5

In this test, we used a switch statement based on a variable initially set to 0. To verify ASPIS's behavior with switch constructs, we created a few cases and set a breakpoint at line 8, immediately before the variable evaluation.

```
(gdb) b 8
(gdb) run
(gdb) set var var=9999
(gdb) continue
```

Test 6

In this test, we dynamically allocated an array on the heap using the malloc() function. We then injected a change into the variable containing the array's address. The objective here was to determine whether ASPIS is capable of detecting pointer corruption referencing dynamically allocated memory on the heap.

```
(gdb) b 25
(gdb) run
(gdb) set var arr=arr+1
(gdb) continue
```

Testing

We compiled each program using the following command:

```
./aspis.sh --llvm-bin /usr/bin --seddi --rasm --suffix 21 -g -o
name_of_compiled_test ~/EmbeddedSystemsProject/name_of_test.c
```

and then ran it in GDB with:

```
gdb name_of_compiled_test
```

For each test, we set a breakpoint, ran the program, and injected a modification into the running code. The specific GDB commands utilized for each individual test are detailed in the respective subsections above.

Results

Across all conducted tests, following the bit-flip injection, ASPIS successfully detected the unauthorized modification. Consequently, the program began looping within the *DataCorruption_Handler()* function. This is the intended behavior, as the function is purposely left empty to allow the developer to implement custom recovery mechanisms. Additionally, we executed the tests without ASPIS protection, compiling the programs with Clang. This comparison was performed to verify that every fault injection we introduced was inherently effective and would naturally propagate in an unprotected environment.

Conclusions

The tests we performed provided clear evidence on the efficacy of the ASPIS compiler protection. By injecting runtime faults into various constructs of the C programming language, ranging from local variables and loop counters to pointers and control flow registers, we observed a consistent and reliable response from the protection mechanism. In every scenario where determinism was appropriately managed to prevent compiler optimization, ASPIS successfully intercepted the unauthorized data modifications.